

Reviewer Pack — Costas Displacement Coincidence Lower-Bounds AC^0 Size for PA...

Entry 6bb89da9ec2e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 02:01:07 UTC

Costas Displacement Coincidence Lower-Bounds AC^0 Size for PARITY

Entry ID: 6bb89da9ec2e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 02:01:07 UTC

1. Conjecture

Field A (mathematical branch): Costas array / Welch–Costas displacement combinatorics (Golomb–Costas radar arrays and B₂/Sidon-deficiency theory of integer sets) — a corner of additive combinatorics developed for radar/sonar that has essentially no presence in circuit complexity literature

Field B (complexity object): AC^0 depth-d circuit size for PARITY_n (Hastad-style lower bounds, viewed through the lens of bottom-gate support designs)

Statement:

Let C be a depth-d AC^0 circuit on n inputs with bottom-level (depth-1) AND/OR gates having literal-supports $S_1, \dots, S_m \subseteq [n]$. Define the Costas displacement defect $\kappa(C) = \log_2(1 + \max_{\delta \in \{1, \dots, n-1\}} \sum_{i=1}^m |\{(a, b) \in S_i \times S_i : a > b \text{ and } a-b \equiv \delta \pmod{n}\}|)$. We conjecture that for every C that computes PARITY_n exactly, $\kappa(C) \geq (1/4) \cdot n^{1/(d-1)}$, and consequently $\text{size}(C) \geq 2^{\Omega(\kappa(C))}$. A single depth-d AC^0 circuit that computes PARITY_n while having $\kappa(C) < (1/4) \cdot n^{1/(d-1)}$ refutes the conjecture.

Rationale (proposer’s reasoning):

PARITY is translation-invariant on the Boolean cube, so any AC^0 circuit computing it must spread its bottom-level supports densely across all cyclic displacements — exactly the property that Costas/Welch arrays minimize. Replacing Hastad’s random restriction by a Costas-style design-theoretic lower bound turns AC^0 PARITY bounds into a question about Sidon-deficiency of support multisets. The invariant depends only on the circuit’s bottom-support combinatorics (not the function’s truth table), so it sidesteps Natural Proofs, and uses no ring operations, sidestepping algebrization.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0670d0ca56916df8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds per (n,d,family) for $n \in \{6,8,10,12,14,16,20,24,28,32,40\}$ and $d \in \{2,3\}$: every tested PARITY circuit must satisfy $\kappa(C) \geq (1/4) \cdot n^{\{1/(d-1)\}}$, with ≥ 0.9 aggregate support fraction over PARITY families and ≥ 0.5 of non-PARITY circuits violating the bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Costas array AC0 circuit complexity parity lower bound - Sidon set bottom gate support AC0 depth circuit parity-displacement difference set switching lemma AC0 parity lower bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.2s

5.1 Generated Python source

```
import sys
import random
import math
import json
from collections import defaultdict

def compute_kappa(supports, n):
    max_displacement = 0
    for S in supports:
        displacement_counts = defaultdict(int)
        for i in range(len(S)):
            for j in range(i + 1, len(S)):
                a, b = S[i], S[j]
                if a > b:
                    delta = a - b
                else:
                    delta = n - (b - a)
                displacement_counts[delta] += 1
        max_displacement = max(max_displacement, sum(displacement_counts.values()))
    return math.log2(1 + max_displacement)

def generate_minterm_dnf(n):
    supports = []
    for i in range(2 ** (n - 1)):
        S = []
```

```

        for j in range(n):
            if (i >> j) & 1:
                S.append(j)
            supports.append(S)
        return supports

def generate_random_dnf(n, size):
    supports = []
    for _ in range(size):
        S = random.sample(range(n), random.randint(1, n))
        supports.append(S)
    return supports

def generate_recursive_sigma3(n):
    if n == 1:
        return [[0]]
    sqrt_n = int(math.ceil(math.sqrt(n)))
    block_size = (n + sqrt_n - 1) // sqrt_n
    supports = []
    for i in range(sqrt_n):
        start = i * block_size
        end = min((i + 1) * block_size, n)
        block_supports = generate_recursive_sigma3(end - start)
        for S in block_supports:
            supports.append([x + start for x in S])
    return supports

def generate_majority_circuit(n):
    supports = []
    for i in range(n):
        S = [i]
        supports.append(S)
    return supports

def generate_and_circuit(n):
    supports = []
    for i in range(n):
        S = [i]
        supports.append(S)
    return supports

def generate_threshold_k_circuit(n, k):
    supports = []
    for i in range(k):
        S = [i]
        supports.append(S)
    return supports

def run_trial(seed):
    random.seed(seed)
    n_values = [6, 8, 10, 12, 14, 16, 20, 24, 28, 32, 40]
    d_values = [2, 3]
    results = []
    instances_tested = 0

    for n in n_values:

```

```

for d in d_values:
    threshold = (1/4) * (n ** (1/(d-1)))

    # Test minterm DNF
    supports = generate_minterm_dnf(n)
    kappa = compute_kappa(supports, n)
    conjecture_holds = kappa >= threshold
    counterexample = "" if conjecture_holds else f"minterm_dnf n={n} d={d} kappa={kappa} < threshold"
    results.append({
        "metric_name": "kappa",
        "metric_value": kappa,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample,
        "seed": seed,
        "n": n,
        "d": d,
        "circuit_type": "minterm_dnf"
    })
    instances_tested += 1

    # Test recursive  $\Sigma 3$ 
    supports = generate_recursive_sigma3(n)
    kappa = compute_kappa(supports, n)
    conjecture_holds = kappa >= threshold
    counterexample = "" if conjecture_holds else f"recursive_sigma3 n={n} d={d} kappa={kappa} < threshold"
    results.append({
        "metric_name": "kappa",
        "metric_value": kappa,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample,
        "seed": seed,
        "n": n,
        "d": d,
        "circuit_type": "recursive_sigma3"
    })
    instances_tested += 1

    # Test random DNF
    supports = generate_random_dnf(n, 2**n)
    kappa = compute_kappa(supports, n)
    conjecture_holds = kappa < threshold
    counterexample = "" if conjecture_holds else f"random_dnf n={n} d={d} kappa={kappa} >= threshold"
    results.append({
        "metric_name": "kappa",
        "metric_value": kappa,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample,
        "seed": seed,
        "n": n,
        "d": d,
        "circuit_type": "random_dnf"
    })
    instances_tested += 1

```

```

# Test majority circuit
supports = generate_majority_circuit(n)
kappa = compute_kappa(supports, n)
conjecture_holds = kappa < threshold
counterexample = "" if conjecture_holds else f"majority_circuit n={n} d={d} kappa={kappa}"
results.append({
    "metric_name": "kappa",
    "metric_value": kappa,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample,
    "seed": seed,
    "n": n,
    "d": d,
    "circuit_type": "majority_circuit"
})
instances_tested += 1

# Test AND circuit
supports = generate_and_circuit(n)
kappa = compute_kappa(supports, n)
conjecture_holds = kappa < threshold
counterexample = "" if conjecture_holds else f"and_circuit n={n} d={d} kappa={kappa}"
results.append({
    "metric_name": "kappa",
    "metric_value": kappa,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample,
    "seed": seed,
    "n": n,
    "d": d,
    "circuit_type": "and_circuit"
})
instances_tested += 1

# Test threshold-k circuit
k = random.randint(1, n)
supports = generate_threshold_k_circuit(n, k)
kappa = compute_kappa(supports, n)
conjecture_holds = kappa < threshold
counterexample = "" if conjecture_holds else f"threshold_k_circuit n={n} d={d} kappa={kappa}"
results.append({
    "metric_name": "kappa",
    "metric_value": kappa,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample,
    "seed": seed,
    "n": n,
    "d": d,
    "circuit_type": "threshold_k_circuit"
})
instances_tested += 1

```

```

return {
    "metric_name": "kappa",
    "metric_value": sum(r["metric_value"] for r in results) / len(results),
    "instances_tested": instances_tested,
    "conjecture_holds": all(r["conjecture_holds"] for r in results),
    "counterexample": next((r["counterexample"] for r in results if not r["conjecture_holds"])),
    "seed": seed
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 240s (returncode 124) without producing a RESULT line, so neither the support nor falsification condition could be evaluated. | next: Reduce the parameter sweep (e.g., cap n at 20 and d=2) or parallelize per-seed evaluation to produce data within the timeout.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	109758
2	preregistration	claude_max	opus	0	0	5583
3	novelty	claude_max	opus	0	0	3693
4	novelty	claude_max	opus	0	0	5788
5	test_gen	mistral	codestral-latest	0	0	310082
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	298393
7	test_gen	mistral	codestral-latest	0	0	310930
8	test_gen	mistral	codestral-latest	0	0	317302
9	judge	claude_max	opus	0	0	4372

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1365902 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 3, 'ollama_remote': 1}

(full prompt+response transcripts available in `research/audit/6bb89da9ec2e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6bb89da9ec2e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6bb89da9ec2e.tar.gz` (if generated)